LEDray_trace: A Lightweight, Multi-Threaded Library for Dynamic Ray Tracing.

Daniel Louk
International Leadership of Texas Garland High School student
Garland, TX, USA
loukdan000@iltexasstudent.org

***Abstract*— Ray tracing is a type of render known for its use across large engines. However, most of these engines lock down their ray tracing code to be difficult to modify. This program demonstrates that multi-threaded real-time ray tracing can be designed to not only allow independent users to modify ray tracing to their specific needs, but to enable future development, sustainability, and scalability.**

***Index Terms*—3D modeling; computer graphics; image processing; multithreading; parallel processing; quaternions; ray tracing; real-time systems; rendering (computer graphics); software architecture; software libraries**

I. INTRODUCTION

This ray tracer exists in order to provide a unique library designed for ease of modification. Under the MIT license [1] and designed with add-ons in mind, the purpose of this program is to give better behind-the-scenes support for developers designing applications utilizing ray tracing. This program is a ray tracing library accompanied by a demonstration program. Ray tracing [2-3] is an algorithm used in computer graphics to show a three dimensional image on the screen by simulating light. Use cases include movies/animation, computer simulations or models (such as those used by NASA), architectural visualization, gaming, etc.

II. USER GUIDE

This section serves to instruct the reader on how to operate the ray-tracer, from an end-point focus. The main library LEDray_trace [4] is equipped with everything necessary to implement a ray tracer into another program, while LEDray_trace_demo [5] is the actual runnable c++ executable given. This section will only cover LEDray_trace_demo and how to use it, as the library is more designed for developers wanting to include ray tracing into their preexisting programs.

A. Importing a Scene

To import a scene, press the 'Import Scene…' button. From there, a popup window will pop up. The user may select a specific color utilizing the radio buttons and RGB color input or set it to use a random color. The color is chosen through a deterministic random function with a constant seed, so with the same number of inputs it should always output the same colors. After selecting 'OK', it will pop up for a .obj [6] file to be selected. This file will contain the object data to be imported. This is the same .obj format blender uses. Currently, it supports only vertex and face definitions, and ignores material, texture, and normal declarations. Faces must be imported in terms of triangles, it requires quads to be broken up. Winding order (the clockwise/counterclockwise orientation a triangle can be defined in) is ignored. The 'Clear Scene' button may be used to clear all objects from the scene.

B. Moving the Camera

The camera can be rotated by moving the mouse. Left/right will move along worldspace Y-axis, and up/down moves along camera up/down directions. The mouse wheel may be used to rotate

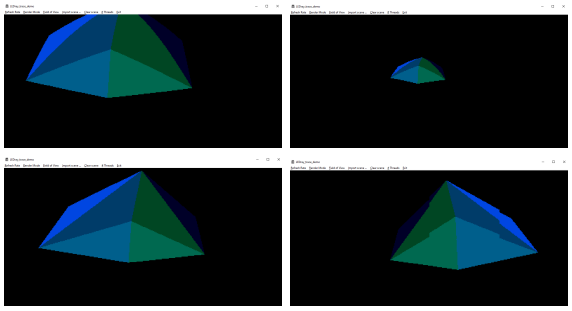

Fig. 1 Curved (top) vs Flat (bottom) rendering at low (right, 90°) and high (left, 270°) FOV values.

about camera-forwards. FOV may be set using the designated 'FOV' button and respective radio dialog. Camera movement is bound to WASD, Shift, and Spacebar. The standard WASD is forwards / left / backwards / right, space is up, shift is down. Note that movement is based on camera space, not world space.

C. Render Mode

Most ray tracers do not allow modification of the rendering mode in the same way that this one can—it may dynamically remap how each frame is mapped. The 'flat' (rectilinear) rendering mode constructs four corners from the set FOV and camera orientation, guaranteed to be the same corners as in 'curved' (equiangle) mode [7]. From there, it linearly interpolates across these points, creating and normalizing its camera directions from there. This means that the camera more closely represents the flat screen it is rendering on. However, this does also mean that FOVs set greater than 180° may result in strange artifacts, such as inverting the screen and looking only behind the camera, as shown in the bottom-right of Fig. 1. In the 'curved' rendering mode, the ray tracer will create equi-angle rays for rendering, enabling larger FOVs without distortion, as shown in the top-right of Fig. 1. However, in order to prevent the distortion from the increase in spatial distance from the position of the screen, it is best recommended for curved monitors. In essence, the 'flat' rendering is designed for screens that do not curve at lower FOV (<180°), and the 'curved' rendering is designed for screens that do curve or for high-FOV models. By ensuring that the actual rays line up with the screen, it can create a more realistically accurate representation by enabling the projection of what one would see if that object actually existed. This flexibility also shows how different rendering options can be supported, and how the program could be modified to support an even wider variety of devices and applications.

D. Refresh Rate

The 'Refresh Rate' button does mean exactly that—it modifies how fast the main runner class will update the screen. If the computer can support faster frames, it may be turned up arbitrarily. Take note that the ray tracer often won't hit the full refresh rate given, as some additional time is taken to perform actions such as repainting the screen, potential update messages, and program latency in general throughout the windows API. The ray tracer itself will only render a frame if it is done with its last frame, so it will only minimally affect performance if the program isn't waiting on repaint for the next scene to be rendered. By default this is set to 5 refresh per second, in order to ensure that initialization doesn't cause excessive lag, especially on lower end devices.

E. Threads Allotment

The ray tracer demo also gives options for how many threads should be used in rendering. By default, it is set to use all logical cores save for two, minimum one. It can also be set to run any other amount of threads. This doesn't guarantee that is how many cores it will run on, it simply determines how many worker threads are created to handle rendering. Setting it equal to or higher than logical thread count is not recommended and may cause systemwide lag/unresponsiveness.

F. System Requirements

Minimum tested requirements is Windows 10 x64 and an Intel Core i5 (or equivalent), 1MB storage and 4GB RAM. It is recommended to have a Quadcore Intel Core i7 or equivalent (or better). The better the CPU the better it will run, as that is the main bottleneck.

III. TECHNICAL ARCHITECTURE

This section will cover more of the LEDray_trace library and how to use it in other projects. It will only use LEDray_trace_demo as examples of how to use features implemented in LEDray_trace.

A. Ray-Triangle intersections

The most important part of any ray tracer is how it determines an intersection between a simple ray and triangle. In LEDray_trace, rays are stored as a point and a vector, and planes are stored in the standard

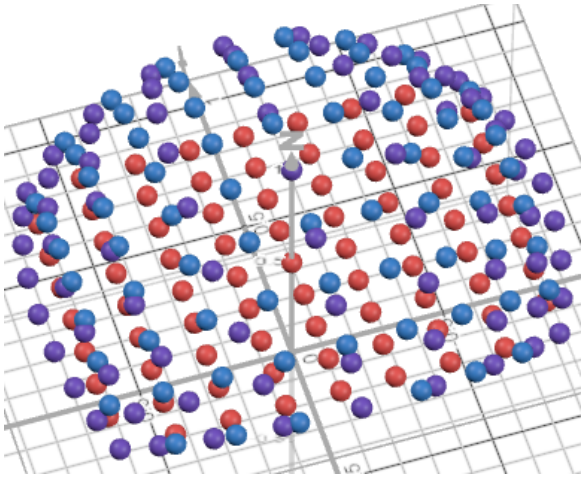

Fig. 2 Camera mappings (Curved vs Flat)

plane equation. Inside the Triangle class intersection, first it calls its superclass intersection method from Plane. This determines the point of intersection and t-value of the intersection between that particular ray and that plane. From there, the Triangle class determines using barycentric coordinates (adjusted for both winding orders) for if the point is inside the triangle. If at any point during this process it is determined that the two cannot intersect or that the triangle is behind the ray, a negative t-value is returned to signify that no intersection exists. This algorithm is known as the Didier Badouel's ray triangle intersection algorithm [8]. Do note, when externally calling these methods, they may return negative t-values. In this case, disregard the point. When returning a negative t-value, it is not guaranteed that the corresponding point in the IntersectionInfoStruct has been initialized or that it contains valid data, as there is no such intersection point.

B. The Camera Class

The Camera class is one of the larger classes in the project, and for good reason. It handles the actual creation of images according to the input data the user would actually change. One (or more) instance(s) of the Camera class is expected to be made in any runner class, each capable of handling their own rotation, position, and independently capture renders of the scene. Each Camera object is designed to have thread-safe rendering capabilities, so render methods can be called in multiple threads. The invalidate method is also important to note, as if a raw operation is performed on the camera, invalidate should be called to set ready to false and have the next frame rebuild necessary camera resources. For example, upon moving the camera all of the rays in the mapping must be rebuilt with new, up-to-date rays according to the new camera position. Additionally, rendering is split into two functions. The render function by itself simply chooses out the rays and handles packaging up the colors for export, while the traceRay function handles the actual rendering calculation of each ray. This is separated so that future versions of the ray tracer may be able to implement recursive designs to handle reflections, refractions, and other optical effects. By default, the Camera has two mappings available, the curved and the flat mapping. As shown in Fig. 2, there is a recreation of the mapping system in desmos. The curved mapping system is in blue, the unnormalized flat mapping system in red , and finally the normalized flat mapping in purple. Do note that the ray tracer normalizes both rays, so the red vectors are only shown to show the intermediary stage before it is normalized. For this reason, both the flat and curved rendering map vectors lay on the same unit circle—their only distinction is the distribution of dots. It may be noted that the purple dots appear more distorted, as they cluster up a bit more along the edges. However, likewise, the angle between pixels on a flat screen to a central observer is also reduced. The interactive representation of this is available at [9].

C. Quaternions and Rotations

Quaternions [10] serve as a very fast and efficient rotation construct [11], designed as axis-half theta rotations. The ray tracer internally only uses quaternions, but may handle either quaternion or euler angle input through built-in conversion methods.

D. Material, Colors, and Image Handling

The library LEDray_trace also comes equipped with its own definition of materials and colors. The base Material class simply holds a plain BGRColor. BGRColor is a struct that holds, in that order, a blue byte, a green byte, and a red byte. This is the standard pixel data structure used by other programs, redefined here for compatibility with other programs, such as the Windows API [12] as used in LEDray_trace_demo. The Camera returns a one dimensional vector of these, from the coordinates passed in left to right, top to bottom (based on default mapping initialization).

Render Times (ms)	Cube Scene (12 faces)	Cube+Pyramid (17 faces)
1 Thread (i7) avg ± 2SE min max	440.434 ± 2.703 402 553	964.574 ± 4.054 865 1118
2 Threads (i7) avg ± 2SE min max	226.332 ± 2.548 200 337	501.222 ± 5.028 422 661
6 Threads (i7) avg ± 2SE min max	144.834 ± 1.593 100 216	254.638 ± 2.390 184 370

Fig. 3 Render Times on an i7 (ms)

F. The Demo

The demo program can be analyzed to glean information about how this library was initially intended to be used. The demo primarily focuses on creating the multithreaded runner threads and displaying results from the ray tracer to the screen. The ray tracer also doesn't handle framerate automatically, so framerates may be handled in any user-defined manner in the end product. In this demo, it calls a render every screen refresh if the previous frame is done calculating. A basic framework for a job queue is created to organize the work loads produced as segments of the screen to render. Each batch is then sent into the Camera class to render, where it locks wherever necessary to be thread-safe for the rest of the program to attempt to call other functions while the frame is rendering, as one would expect from such an architecture that continuously renders when a user wants input.

G. Performance

The base ray tracer is not yet fully optimized, mostly in terms of scalability with large data. Currently, the render itself is O(pixels x scene). When BVH [13] is implemented, this would be reduced to O(pixels x log2(scene)), which would be much more scalable with large scenes. As seen in Fig. 3, the render times agree with this. Render times were taken over 500 frames, with the first 21 frames discarded. Frame delay was set to 20ms on program startup to reduce waiting on internal program latency. Do note that there are some diminishing returns, as the demo program introduces some additional overhead. In this overhead, ray tracing calculations may not be running, increasing render times. This is where the 100ms delay and 200ms delay exact frames come from. This also demonstrates the trade-off between faster refresh rates that consume more overhead or slower refresh rates that may leave the computer waiting for the next batch of instructions. This is also why the scaling with more cores is more pronounced with a more complex scene, as there is less wait time.

IV. FUTURE WORK

This project does leave plenty of work left to do for a professional-level ray tracer. For example, it would greatly benefit from having an additional accelerations structure, such as BVH (Bounding Volume Hierarchy) [13]. The ray tracer is specifically designed to support such addons in the future, with proper sustainability practices in mind. Additional methods for subclass support and sustainability was a significant focus for the project. Architecture such as BVH could be implemented through a subclass of the Camera class, reflections and refractions are able to be determined through subclasses of the Material class, and additional objects can be created for optimization reasons through subclasses of the Plane class (they are not forced to call the superclass intersection function, they may completely override it if they operate differently).

V. CONCLUSION

Overall, this project was designed partially as a proof-of-concept, demonstrating how a ray tracer can be designed to be more open to the user. This ray tracer lays a foundation for other similar projects and/or ray tracing programs to utilize and build off of, not necessarily as a complete project in and of itself. In other words, this library is a library. It is designed as such—to be used in another program. This can allow programmers to import in a ray tracer, then create any other part of a project they want, and be able to interface through to the ray tracer to render what they want, without the need to create their own or too tightly curtail to a specific, unmodifiable API, as this program is designed to be easy to modify.

REFERENCES

[1] “The MIT License,” Open Source Initiative, https://opensource.org/license/mit (accessed Apr. 2026).

[2] Some techniques for shading machine renderings of Solids | Proceedings of the April 30--May 2, 1968, Spring Joint Computer Conference, https://dl.acm.org/doi/10.1145/1468075.1468082 (accessed Apr. 2026).

[3] Ray Tracing in one weekend series, https://raytracing.github.io/books/RayTracingInOneWeekend.html (accessed 2026).

[4] FireDrake987, “Firedrake987/LEDray_trace,” GitHub, https://www.github.com/FireDrake987/LEDray_trace (accessed Apr. 2026).

[5] FireDrake987, “Firedrake987/LEDray_trace_demo,” GitHub, https://www.github.com/FireDrake987/LEDray_trace_demo (accessed Apr. 2026).

[6] The OBJ file format, https://www.scratchapixel.com/lessons/3d-basic-rendering/obj-file-format/obj-file-format.html (accessed Apr. 2026).

[7] Jian Xu SUSTech Southern University of Science and Technology ShenZhen, A comprehensive overview of fish-eye camera distortion correction methods, https://arxiv.org/html/2401.00442v2 (accessed Apr. 2026).

[8] “Didier Badouel’s ray-triangle intersection algorithm,” Didier Badouel’s Ray-Triangle Intersection Algorithm · Lightbox, https://ton.github.io/lightbox/2015/04/27/didier-badouel-ray-triangle-intersection.html#:~:text=Didier%20Badouel’s%20Ray%2DTriangle%20Intersection%20Algorithm%20%C2%B7%20Lightbox (accessed Apr. 2026).

[9] “3D mappings,” Desmos, https://www.desmos.com/3d/uzjjrain8j (accessed Apr. 2026).

[10] On quaternions, or on a new system of imaginaries in Algebra by, https://www.maths.tcd.ie/pub/HistMath/People/Hamilton/OnQuat/OnQuat.pdf (accessed Apr. 2026).

[11] Animating rotation with quaternion curves | ACM SIGGRAPH Computer Graphics, https://dl.acm.org/doi/10.1145/325165.325242 (accessed Apr. 2026).

[12] Drewbatgit, “Programming reference for the win32 API - win32 apps,” Win32 apps | Microsoft Learn, https://learn.microsoft.com/en-us/windows/win32/api/ (accessed Apr. 2026).

[13] I. Wald, S. Boulos, and P. Shirley, Ray Tracing Deformable Scenes using Dynamic Bounding Volume Hierarchies, https://www.sci.utah.edu/~wald/Publications/2006/BVH/download/dynbvh.pdf (accessed Apr. 2026).